Report
Task 1.1
Used Method

The script processes the corrupted audio signal through the following sequential pipeline:

- **Frequency Restriction (Bandpass):** A 5th-order Butterworth bandpass filter is first applied to restrict the audio to a standard speech frequency range (40 Hz to 2900 Hz). This broadly eliminates extreme high and low-frequency noise outside the human vocal range.
- **Targeted Noise Removal (Notch Filtering):** The script iterates through a pre-calculated list of noise spike frequencies (`peak_frequencies`). For each targeted frequency, an IIR (Infinite Impulse Response) notch filter with a quality factor (Q) of 5.0 is applied.
- **Zero-Phase Filtering:** The `filtfilt` function is utilized to apply the filters both forwards and backwards. This ensures zero phase distortion, keeping the speech aligned.
- **Audio Normalization:** The final filtered data array is mathematically normalized by dividing it by its maximum absolute amplitude to prevent clipping and ensure an audible volume level upon playback.

Final Result

I know its probably wrong, but the best I could get was "رايح الكلية"

Task 1.2
Detection of Red and Blue Balls

To identify the balls, the system relies on color masking. First, the images are converted from the standard BGR format into the HSV (Hue, Saturation, Value) color space. Using the `cv2.inRange()` function, specific hue ranges are defined for the colors blue and red. This isolates the target colors into binary masks, where pixels matching the colors become white, and the rest of the image becomes black. Because red wraps around the HSV spectrum, two separate masks are combined to capture all shades of red.

Handling Lighting Conditions and Shadows

Real-world images present challenges such as shadows and varying lighting. Converting the image to the HSV color space is the primary method used to handle these conditions. Unlike the BGR color space which mixes color and brightness, the HSV color space separates the pure color (Hue) from the lighting intensity (Value). This allows the color thresholds to remain accurate whether the ball is located in bright sunlight or hidden in a deep shadow.

Noise Reduction and False Detection Mitigation

To ensure accuracy and minimize false positives (objects incorrectly detected as balls), the system applies filtering to the color masks:

- **Area Filtering:** The `cv2.findContours()` function extracts all colored blobs. Blobs with a contour area smaller than 50 pixels are discarded to remove background noise and visual disturbances.
- **Aspect Ratio Check:** To prevent long, rectangular background objects of similar colors from being flagged, an aspect ratio check is applied. A bounding box is calculated for each blob, and only those with a roughly square aspect ratio (between 0.5 and 2.0) are processed further.
- **Morphological Solidification (Convex Hull):** The black patterns on the balls cause the color masks to fragment. To fix this, `cv2.convexHull()` wraps a boundary around the fragmented pieces, and `cv2.drawContours()` fills them in. This creates a solid, unified shape for the detector to read, preventing false negatives.

Determining Location and Size

Once the shapes are solidified and a light Gaussian blur is applied to smooth the edges, the location and size of each ball are determined using the `cv2.HoughCircles` algorithm. This function analyzes the cleaned mask to detect circular geometries. For each validated circle, the algorithm outputs the center coordinates `(x_center, y_center)`, which represent the ball's location. The size (width and height) is calculated by multiplying the detected circle's radius by two.

Generating the Final Label Files

The coordinates and dimensions extracted are then normalized (represented as values between 0.0 and 1.0) by dividing them by the original image's total width and height. A list of detections is formatted into strings following the required structure: `<class_id> <x_center> <y_center> <width> <height>`. The `<class_id>` is assigned as `0` for Blue balls and `1` for Red balls. Finally, the system automatically writes these strings into individual `.txt` files that share the exact same name as their corresponding input images

.